Hierarchical Multi-View Graph Pooling With Structure Learning

Zhen Zhang, Jiajun Bu, *Member, IEEE*, Martin Ester, *Senior Member, IEEE*, Jianfeng Zhang, Zhao Li, Chengwei Yao, Huifen Dai, Zhi Yu, and Can Wang, *Member, IEEE*

Abstract—Graph Neural Networks (GNNs), whch generalize deep neural networks to graph-structured data, have drawn considerable attention and achieved state-of-the-art performance in numerous graph related tasks. However, existing GNN models mainly focus on designing graph convolution operations. The graph pooling (or downsampling) operations, that play an important role in learning hierarchical representations, are usually overlooked. In this paper, we proposed a novel multi-view graph pooling operator dubbed as MVPool, which ranks nodes across different views with different contextual graph information. Meanwhile, attention mechanism is utilized to promote the collaboration of different views for generating robust node rankings. Then the pooling operation adaptively selects a subset of nodes to form an induced subgraph based on the ranking list. To preserve the underlying graph topological information, we further introduce a structure learning mechanism to learn a refined graph structure for the pooled graph at each layer. The proposed MVPool operator is a general strategy that can be integrated into various graph neural network architectures, including GCN, GAT and GraphSAGE, etc. By combining MVPool operator with graph neural networks, we perform hierarchical representation learning for both node and graph level classification as well as clustering tasks. Experimental results on thirteen widely used transductive and inductive benchmarks demonstrate the effectiveness of our proposed model.

Index Terms—Graph neural networks, graph pooling, deep learning, node classification, graph classification, clustering

1 Introduction

Deep neural networks with convolutional and pooling layers have achieved great success in various challenging tasks, ranging from computer vision [1], [2], natural language understanding [3], [4] to video processing [5], [6]. The addition of pooling layers after convolutional layers can scale down the size of inputs and enlarge the receptive fields, which gives rise to hierarchical representation learning. The data in these aforementioned tasks are typically represented in the euclidean space (i.e., modeled as 2-D or 3-D tensors), thus usually containing locality and order information for the convolution operations [7]. However, in many real-world problems, a large amount of data, such as social networks [8], chemical molecules [9] and biological networks [10], are lying on non-Euclidean domains that can be naturally represented as graphs. Due to the neural network's powerful capabilities, it's quite appealing to generalize the convolution and pooling operations to graph-structured data.

Recently, there have been a myriad of attempts to generalize the convolution operations to arbitrarily-structured graphs, referred to as graph neural networks (GNNs for short). In general, these algorithms can be classified into two big categories: spectral and spatial approaches. For the spectral methods, they typically define the graph convolution operations based on graph Fourier transform [7], [11], [12], [13], [14]. For the spatial methods, the graph convolution operations are devised by aggregating the node representations directly from its neighborhood [15], [16], [17], [18], [19]. Majority of the aforementioned methods mainly involve transforming, propagating and aggregating node features across the graph, which can fit in the message passing scheme [20]. GNNs have been applied to different types of graphs [17], [21], and obtained outstanding performance in numerous graph related tasks, including node classification [12], [15], graph classification [22], [23], [24], link prediction [25], [26] and recommendation [27], [28], *etc*.

Nevertheless, the pooling operation in graphs have not been extensively studied yet, though they act a pivotal part in learning hierarchical representations. Existing GNN models usually pass messages between nodes through directly connected edges, thus the aggregation procedure of k iterations will make use of k-hop neighborhood information for each node [19]. Due to the lack of pooling mechanism, it has

- *Zhen Zhang, Jiajun Bu, Chengwei Yao, and Can Wang are with the Zhejiang Provincial Key Laboratory of Service Robot, College of Computer Science, Zhejiang University, Hangzhou, Zhejiang 310027, China, and also with the Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Hangzhou, Zhejiang 310058, China. E-mail: {zhen_zhang, bjj, yaochw, wcan}@zju.edu.cn.*
- *Martin Ester is with the School of Computing Science, Simon Fraser University, Vancouver, BC V6B 5K3, Canada. E-mail: ester@sfu.ca.*
- *Jianfeng Zhang is with the College of Computer Science, Zhejiang University, Hangzhou, Zhejiang 310027, China, and also with Alibaba Group, Hangzhou, Zhejiang 311121, China. E-mail: zjf2308@alibaba-inc.com.*
- *Zhao Li is with Alibaba Group, Hangzhou, Zhejiang 311121, China. E-mail: lizhao.lz@alibaba-inc.com.*
- *Huifen Dai is with the Fourth Affiliated Hospital, Zhejiang University School of Medicine, Yiwu, Zhejiang 322000, China. E-mail: daihuifen@zju.edu.cn.*
- *Zhi Yu is with the School of Software Technology, Zhejiang University, Ningbo, Zhejiang 315100, China, and also with Ningbo Research Institute, Zhejiang University, Ningbo, Zhejiang 315100, China. E-mail: yuzhirenzhe@zju.edu.cn.*

Manuscript received 12 June 2020; revised 2 Apr. 2021; accepted 3 June 2021. Date of publication 18 June 2021; date of current version 7 Dec. 2022. (Corresponding authors: Jiajun Bu and Zhao Li.) Recommended for acceptance by P. Bogdanov. Digital Object Identifier no. 10.1109/TKDE.2021.3090664

1041-4347 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

been observed that using deeper GCN layers [12] combined with residual connections [2] perform even worse than a 2-layer version for semi-supervised node classification task. This phenomenon can be explained as the Laplacian over-smoothing issue[29], which leads to indistinguishable node representations as the GNN model goes deeper. Moreover, it will be even more problematic for graph classification task when the model has difficulty in obtaining global hierarchical information [22]. The goal of graph classification is to predict the label associated with the entire graph by utilizing both node features and graph structure information, i.e., a graph level representation is needed. GNNs [30] are originally designed to learn meaningful node level representations, thus a commonly adopted approach to generate graph level representation is to globally summarize (e.g., mean-pool or max-pool) all the node representations in the graph. Although workable, the graph level representation generated via this way is inherently "flat", since the entire graph structure information is neglected during this process. Meanwhile, graphs often have different substructures and nodes are of different roles, therefore they should contribute differently to the learned graph level representation. For example, in the protein-protein interaction graphs, the certain substructures may represent some specific functionalities, which are of great significance to predict the whole graph characteristics. To enlarge the receptive field and capture graph's hierarchical global information, a hierarchical pooling procedure is demanded.

There exists some very recent work that focuses on hierarchical pooling in graph neural networks [22], [31], [32], [33], [34], [35]. These models usually coarsen the graphs through grouping or sampling nodes into subgraphs level by level, thus the entire graph information is gradually reduced to hierarchical induced subgraphs. However, the designing of graph pooling operations is still at its early stage and can be further improved. In node grouping approaches, the hierarchical pooling methods [22], [32], [34], [36] suffer from high computational complexity, which require additional neural networks to cluster and then downsize the nodes. In node sampling approaches, the induced subgraph, which is sampled according to the calculated node ranking list [31], [33], [35], might fail to preserve the underlying substructures and eventually lose the completeness of graph topological information. For instance, two nodes that are not directly connected but sharing many common neighbors in the original graph might become unreachable from each other in the induced subgraph, even if intuitively they ought to be "close" in the subgraph. Therefore, the distorted graph structure will hinder the message passing in subsequent layers. Furthermore, various kinds of measurements have been proposed to rank nodes in the graph pooling procedure, such as scalar projection value in [31], self-attention weight in [35], node information score in [37]. As each individual view of ranking measurement is usually biased [38], [39] and the information in different views can be very different, it is of great necessity to learn robust node rankings with the collaboration of multiple views.

To address the aforementioned limitations, we propose a novel multi-view graph pooling operator MVPool with structure learning mechanism to learn hierarchical representations in graph. More specifically, we first introduce a set of measurements to evaluate the importance of nodes in different views with different contextual graph information. The multiple view-specific node importance scores are then dynamically aggregated for generating robust node ranking list via neural attention mechanism [4]. Then, the pooling operation adaptively selects a subset of top-ranked nodes according to our obtained node ranking list, which forms an induced subgraph. Next, we apply a structure learning mechanism with sparse attention [40] to the pooled graph, aiming to learn a refined graph structure that preserves the underlying substructures in the original graph. We integrate the pooling operator into graph convolutional neural network to perform both node and graph level classification as well as clustering tasks. The whole learning procedure can be optimized in an end-to-end manner. To summarize, the main contributions of this paper are as follows:

- We introduce a multi-view graph pooling operator MVPool that can be integrated into various graph neural network architectures. The collaboration of multiple views has the advantage of generating robust node ranking list for pooling operation.
- To the best of our knowledge, we are the first to design structure learning mechanism for the pooled graph, which empowers the capability of learning a refined graph structure to well preserve the graph's underlying substructures.
- We conduct extensive experiments on nine public datasets including node and graph level tasks under both transductive and inductive settings to demonstrate MVPool's effectiveness as well as superiority compared with a range of state-of-the-art methods.

2 Related Work

2.1 Graph Neural Networks

Mining knowledge in graphs has attracted a surge of interests in both academic and industrial communities. Many methods have been proposed to learn low dimensional node representations, which could preserve both the graph topological and node attribute information. For instance, DeepWalk [41] and node2vec [42] perform random walks on graph to generate node sequences, which are regarded as word sequences and then fed into skip-gram model [43] to learn latent representations. LINE [44] utilizes both first and second order relationships to capture graph's structure proximities. This type of approaches try to "linearize" the graph structure into node sequences via random walks. Then, skip-graph model [43], [45] or matrix factorization [46] techniques are employed to model the co-occurrence statistics in these generated node sequences. However, random walk based methods can not be trained in an end-to-end fashion, thus usually achieving unsatisfied performance in the downstream tasks.

Recently, graph neural networks (GNNs) have successfully generalized convolutional operation to graph structured data and achieved state-of-the-art performance in numerous graph related tasks. Existing GNN models can be roughly categorized into two big branches: spectral and spatial approaches. The spectral methods typically define the parameterized filters according to graph spectral theory. [11]

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

first proposed to define convolution operations for graph in the Fourier transform domain. Due to its heavy computation cost, it has difficulty in scaling to large graphs. ARMA[13] provided a recursive and distributed filter that can be efficiently trained. Later on, [7] improved its efficiency by approximating the K-polynomial filters through Chebyshev expansion. GCN [12] further simplified the ChebNet by truncating the Chebyshev polynomial to the first-order approximation of the localized spectral filters. We choose GCN as our model's building block and briefly review its mechanism here. For the kth layer in GCN, it takes graph $\mathcal{G}$'s adjacent matrix $\mathbf{A}$ and hidden representation matrix $\mathbf{H}_k$ as input, then the next layer's output will be generated as follows:

$$\mathbf{H}_{k+1} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}_k\mathbf{W}^k), \tag{1}$$

where $\sigma(\cdot)$ is the non-linear activation function and $\mathbf{H}_0 = \mathbf{X}$, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacent matrix with self-connections. $\tilde{\mathbf{D}}$ is the diagonal degree matrix of $\tilde{\mathbf{A}}$, and $\mathbf{W}^k \in \mathbb{R}^{d_k \times d_{k+1}}$ is a trainable weight matrix. Please note that our proposed MVPool operator can also be integrated into other graph neural network architectures. We will discuss this in the experimental section.

The spatial approaches design convolution operations by directly aggregating the node's neighborhood information. Among them, GraphSAGE [15] proposed an inductive algorithm that can generalize to unseen nodes by aggregating its neighborhood content information. GAT [17] utilized attention mechanism to aggregate nodes' neighborhood representations with different weights. SGC [47] simplified GCN [12] through removing the nonlinearities and merging multiple parametric matrices into a single matrix. More details can be found in several comprehensive surveys on graph neural networks [48], [49], [50]. Nevertheless, the above mentioned two branches of GNNs are mainly designed for learning meaningful node representations, and unable to generate hierarchical graph representations due to the lack of pooling operations.

2.2 Graph Pooling

Pooling operations in conventional Convolutional Neural Networks (CNNs) can reduce the size of input feature maps and the number of training parameters. Traditional pooling operator (e.g., 2×2 max-pooling kernel) in CNNs cannot be directly applied to irregular graph structured data. To bridge this gap, several graph pooling approaches are proposed. DiffPool [22] proposed to softly assign nodes to a set of clusters using neural networks, which forms a dense cluster assignment matrix and is computation expensive. gPool [31] and SAGPool [35] devised a top-K node selection procedure to form an induced subgraph for the next input layer. Though efficient, it might lose the completeness of the graph structure information and result in isolated subgraphs, which will hamper the message passing process in subsequent layers. [51] integrates DiffPool [22] and SAGPool [35] into a unified model. EdgePool [32] designed pooling operation by contracting the edges in the graph, but its flexibility is poor because it will always pool roughly half of the total nodes. iPool [52] presented a parameter-free pooling scheme which is invariant to graph isomorphism. EigenPool [34] introduced a pooling operator based on the graph Fourier transform, which controls the pooling ratio through spectral clustering and it's also very time consuming.

In addition, there are also some approaches that perform global pooling. For instance, Set2Set [53] implemented the global pooling operation by aggregating information through LSTMs [54]. DGCNN [55] pooled the graph according to the last channel of the feature map values, which are sorted in the descending order. Graph topological based pooling operations are proposed in [7], [56] and [57] as well, where Graclus [58] and graph coarsening methods [59] are employed as a pooling module. However, the vast majority of existing methods only utilize limited graph contextual information, ignoring the collaboration and contribution of different information sources.

3 The Proposed Model

3.1 Notations and Problem Formulation

For an arbitrary graph $\mathcal{G}_i = (\mathcal{V}_i, \mathcal{E}_i, \mathbf{X}_i)$, we have n_i and e_i denote the number of nodes and edges, respectively. Let $\mathbf{A}_i \in \mathbb{R}^{n_i \times n_i}$ be the adjacent matrix describing its edge connection information and $\mathbf{X}_i \in \mathbb{R}^{n_i \times f}$ represents the node feature matrix, where f is the dimension of node attributes. Since the graph structure and node numbers change between layers due to the graph pooling operation, we further represent the ith graph fed into the kth layer as $\mathcal{G}_i^k$ with n_i^k nodes. The adjacent matrix and hidden representation matrix are then denoted as $\mathbf{A}_i^k \in \mathbb{R}^{n_i^k \times n_i^k}$ and $\mathbf{H}_i^k \in \mathbb{R}^{n_i^k \times d}$. Label matrix $\mathbf{Y} \in \mathbb{R}^{n \times c}$ indicates the associated labels for each node or each graph which depends on the corresponding tasks, e.g., if v_i (or $\mathcal{G}_i$) belongs to class j, then $\mathbf{Y}_{ij} = 1$, otherwise $\mathbf{Y}_{ij} = 0$. With the above notations, we formally define our problem as follows:

Input: Given a set of graph data $G = \{\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_n\}$ (e.g., $n = 1$ in node level task and $n > 1$ in graph level task) with its label information $\mathbf{Y}_L$ as supervision, the number of graph neural network layers K, pooling ratio r, and representation dimension d in each layer.

Output: For classification task, our goal is to predict the unknown node labels and graph labels with graph neural network in an end-to-end way. For clustering task, we utilize the learned node-level and graph-level representations as inputs for widely adopted clustering algorithms.

3.2 Multi-View Graph Pooling Operation

In this subsection, we introduce our proposed graph pooling operation to enable down-sampling on graph data. Inspired by [31], [35], the pooling operation identifies a subset of important nodes to form a new but smaller graph. Here, we devise a novel multi-view graph pooling operation, which promotes the collaboration of different views for generating robust node rankings.

The key of sampling based graph pooling operation is to define a criterion that provides a ranking which measures the importance of nodes. There exist a wide range of views to interpret the term "node importance", which depends on the utilization of different graph contextual information. In general, real-world graphs usually contain rich graph structure and node feature information. Therefore, we introduce a set of *view-specific* measurements to evaluate the

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

importance of each node based on its specific graph contextual information:

Structure Specific View. The graph structure itself can provide useful information on reflecting the importance of nodes, such as node degrees and the shortest paths between different nodes, *etc*. Hence, we borrow the definition of node centrality to depict their importance, which has been extensively studied in graph theory. Specifically, we choose the simplest degree centrality to calculate the node importance score, because other centrality based measurements like closeness centrality [60], betweenness centrality [61] and eigenvector centrality [62] are computation expensive. Also, strictly following the definition of node degree centrality will result in large integer values (e.g., degree(v) = 20 if node v has 20 neighbors) and it does not match the scales generated in other views which may even dominate the final rankings, we therefore employ a scaled and shifted node degree centrality as follows:

$$\mathbf{p}^{\mathrm{s}} = \sigma(\alpha * log(\mathrm{d}egree(\mathbf{A}_i^k) + \epsilon) + \beta), \tag{2}$$

where $\mathbf{A}_i^k$ is the adjacent matrix and degree($\cdot$) returns the node degrees by counting the number of links. ϵ is a small positive constant (e.g., 10^{-6}) in logarithm. α and β are two trainable scalar parameters for scaling and shifting the degree centrality. We use a non-linear activation function $sigmoid(\cdot)$ to transform the values into range [0,1], and $\mathbf{p}^{\mathrm{s}} \in \mathbb{R}^{n_i^k}$ encodes the node importance scores at kth layer.

Feature Specific View. Apart from topological information, graphs are usually associated with abundant features and node features can largely describe its properties. For example, the node feature information in chemical molecules reflects the types of atoms, which is important for predicting the graph characteristics. Thereupon, it is an effective resource to express the importance of node within a graph. We directly calculate the node importance score from node feature with a Multi-Layer Perceptron (MLP) as follows:

$$\mathbf{p}^{\mathrm{f}} = \sigma(MLP(\mathbf{H}_i^k)), \tag{3}$$

where $\mathbf{H}_i^k$ is the node representation matrix at kth layer. MLP($\cdot$) consists of several fully connected layers with the output dimension setting as 1. Then, we have $\mathbf{p}^{\mathrm{f}} \in \mathbb{R}^{n_i^k}$ represent their importance scores extracted from feature information.

Structure and Feature Specific View. To make more comprehensive and objective node evaluation, it is necessary to make full use of both graph structure and node feature information. Existing structure and feature based graph pooling approaches [35], [51], which capture only local information of a graph with one layer graph convolutional network [12], might lose the global perspective of the graph during the ranking procedure and then lead to biased results. Hence, we propose to exploit PageRank variants [63], [64] to explore node's global perspectives within a graph. In details, we first utilize both graph structure and node feature information to map each node into a real-value with range in [0,1], which is achieved by a graph neural network mapping function. Then, the scores are propagated via RageRank scheme until convergence. The mathematical formulations are defined as follows:

$$\mathbf{PR} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}_i^k\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}_i^k\mathbf{\Theta})\mathbf{p}^{sf} = \sigma(\gamma(\mathbf{I} - (1-\gamma)\tilde{\mathbf{A}}_i^k)^{-1}\mathbf{PR}), \tag{4}$$

where $\tilde{\mathbf{D}}$ is the diagonal degree matrix of $\tilde{\mathbf{A}}_i^k$, and $\mathbf{\Theta} \in \mathbb{R}^{d_k \times 1}$ is a trainable weight matrix. $\mathbf{I}$ is an identity matrix and γ is the restart probability with constraint $\gamma \in (0, 1]$. It is worthy noting that directly calculating Eq. (4) can be computation expensive due to the inverse operation, thus we utilize power iteration to approximate the PageRank values [65]. Similarly, we use $\mathbf{p}^{sf} \in \mathbb{R}^{n_i^k}$ to indicate their importance scores in the view of structure and feature information.

As individual views usually result in biased evaluation and the importance of each view can be quite different, we propose to promote the collaboration of different views for generating robust node rankings. To this end, we try to automatically learn the weights for each *view-specific* output $\{\mathbf{p}^c\}_{c=1}^C$ (C is the set of views). Inspired by recent advance of neural attention [4], we define the weight of view c as follows:

$$w^c = \frac{exp(\sigma(\mathbf{v}^\top \mathbf{z}_c + \mathbf{b}_c))}{\sum_{k=1}^C exp(\sigma(\mathbf{v}^\top \mathbf{z}_k + \mathbf{b}_k))}, \tag{5}$$

where $\mathbf{v}$ is the concatenation of all different views $\{\mathbf{p}^c\}_{c=1}^C$, $\mathbf{z}_c$ and $\mathbf{b}_c$ are the parameter vectors of view c. Thus, the larger the weight w^c is, the more important the view c is. With the weights as coefficients, the final robust node importance score that jointly consider multiple specific views can be generated as follows:

$$\mathbf{p} = \sum_{c=1}^{C} w^c \cdot \mathbf{p}^c. \tag{6}$$

After having obtained the final node importance score, we then employ them to select nodes that should be preserved by the pooling operator. In details, we first rank the nodes according to their importance scores, then a subset of top-ranked nodes are selected as follows:

$$\begin{aligned} \mathrm{i}dx &= \text{top-rank}(\mathbf{p}, \lceil r * n_i^k \rceil) \\ \tilde{\mathbf{p}} &= sigmoid(\mathbf{p}(\mathrm{i}dx)) \\ \tilde{\mathbf{H}}_i^{k+1} &= \mathbf{H}_i^k(\mathrm{i}dx, :) \odot (\tilde{\mathbf{p}}^\top \mathbf{1}) \\ \mathbf{A}_i^{k+1} &= \mathbf{A}_i^k(\mathrm{i}dx, \mathrm{i}dx), \end{aligned} \tag{7}$$

where r is the pooling ratio and top-rank($\cdot$) denotes the function that returns the indices of the top $n_i^{k+1} = \lceil r * n_i^k \rceil$ values. $\mathbf{H}_i^k(\mathrm{i}dx, :)$ and $\mathbf{A}_i^k(\mathrm{i}dx, \mathrm{i}dx)$ perform the row or (and) column extraction to form the node representation matrix and adjacent matrix for the induced subgraph, $\mathbf{1}$ is a matrix with all elements as 1. The gate operation of multiplying $\tilde{\mathbf{p}}$ make it possible to perform back-propagation in an end-to-end manner. Thus, we have $\tilde{\mathbf{H}}_i^{k+1} \in \mathbb{R}^{n_i^{k+1} \times d}$ and $\mathbf{A}_i^{k+1} \in \mathbb{R}^{n_i^{k+1} \times n_i^{k+1}}$ represent the node feature and graph structure information of next layer.

3.3 Structure Learning Mechanism

In this subsection, we present how our proposed structure learning mechanism learns a refined graph structure in the pooled graph. As we have illustrated in previous sections,

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

the pooling operation might result in highly related nodes being disconnected in the induced subgraph, which loses the completeness of the graph structure information and further hinders the message passing procedure (also see the toy example in Fig. 2). Meanwhile, the graph structure obtained from domain knowledge (e.g., social network) or established by human (e.g., KNN graph) are usually non-optimal for the learning task in graph neural networks, due to the lost or noisy information. To overcome this problem, [66] proposed to adaptively estimate graph Laplacian using an approximate distance metric learning algorithm, which might lead to local optimal solution. [67] introduced to learn the constructed graph structure for node label estimation, however it generates dense connected graph and is not applicable in the hierarchical graph representation learning scenario.

Here, we develop a novel structure learning layer, which learns sparse graph structure through sparse attention mechanism [40]. For graph $\mathcal{G}_i$'s pooled subgraph $\mathcal{G}_i^k$ at its kth layer, we take its structure information $\mathbf{A}_i^k \in \mathbb{R}^{n_i^k \times n_i^k}$ and hidden representations $\mathbf{H}_i^k \in \mathbb{R}^{n_i^k \times d}$ as input. Our target is to learn a refined graph structure that encodes the underlying pairwise relationship between each pair of nodes. Formally, we utilize a single layer neural network parameterized by a weight vector $\vec{\mathbf{a}} \in \mathbb{R}^{1\times 2d}$. Then, the similarity score between node v_p and v_q calculated by the attention mechanism can be expressed as:

$$\mathbf{E}_i^k(p,q) = \sigma(\vec{\mathbf{a}}\,[\mathbf{H}_i^k(p,:)||\mathbf{H}_i^k(q,:)]^\top) + \lambda \cdot \mathbf{A}_i^k(p,q), \tag{8}$$

where $\sigma(\cdot)$ is the activation function like $ReLU(\cdot)$ and $||$ represents the concatenation operation. $\mathbf{H}_i^k(p,:) \in \mathbb{R}^{1\times d}$ and $\mathbf{H}_i^k(q,:) \in \mathbb{R}^{1\times d}$ indicate the pth and qth row of matrix $\mathbf{H}_i^k$, which denote the representations of node v_p and v_q, respectively. Specifically, $\mathbf{A}_i^k$ encodes the induced subgraph structure information, where $\mathbf{A}_i^k(p,q) = 0$ if node v_p and v_q are not directly connected. We incorporate $\mathbf{A}_i^k$ into our structure learning layer to bias the attention mechanism to give a relatively larger similarity score between directly connected nodes, and at the same time try to learn the underlying pairwise relationships between disconnected nodes. λ is a trade-off parameter between them.

To make the similarity score easily comparable across different nodes, we normalize them across nodes using the softmax function:

$$\mathbf{S}_i^k(p,q) = \frac{exp(\mathbf{E}_i^k(p,q))}{\sum_{m=1}^{n_i^k} exp(\mathbf{E}_i^k(p,m))}. \tag{9}$$

However, the softmax transformation always has non-zero values and thus results in dense fully connected graph, which may introduce lots of noise into the learned structure. Hence, we propose to utilize sparsemax function [40], [68], which retains most the important properties of softmax function and has in addition the ability of producing sparse distributions. The $sparsemax(\cdot)$ function aims to return the Euclidean projection of input onto the probability simplex [69] and can be formulated as follows:

$$\begin{aligned} &\mathbf{S}_i^k(p,q) = sparsemax(\mathbf{E}_i^k(p,q)) \\ &sparsemax(\mathbf{E}_i^k(p,q)) = [\mathbf{E}_i^k(p,q) - \tau(\mathbf{E}_i^k(p,:))]_+, \end{aligned} \tag{10}$$

where $[x]_+ = \max\{0,x\}$, and $\tau(\cdot)$ is the threshold function that returns a threshold according to the procedure shown in Algorithm 1. Thus, $sparsemax(\cdot)$ preserves the values above the threshold and the other values will be truncated to zeros, which brings sparse graph structure. Similarly to softmax function, $sparsemax(\cdot)$ also has the properties of non-negative and sum-to-one, that's to say, $\mathbf{S}_i^k(p,q) \geq 0$ and $\sum_{q=1}^{n_i^k} \mathbf{S}_i^k(p,q) = 1$. In what follows, we provide the proof of correctness for Algorithm 1.

Algorithm 1. The Calculation Procedure of $sparsemax(\cdot)$

Require: Input vector $\mathbf{z} \in \mathbb{R}^n$.
1: Sort $\mathbf{z}$ into $\mathbf{u}$: $u_1 \geq u_2 \geq \cdots \geq u_n$.
2: Get $\rho = \max\{1 \leq j \leq n : u_j + \frac{1}{j}(1 - \sum_{i=1}^{j} u_i) > 0\}$.
3: Define $\tau(\mathbf{z}) = \frac{1}{\rho}(\sum_{i=1}^{\rho} u_i - 1)$.
4: Return $\mathbf{x}$ s.t. $x_i = \max\{z_i - \tau(\mathbf{z}), 0\}, i = 1,\ldots,n$.

A Proof of Correctness for Algorithm 1. To summarize, $sparsemax(\cdot)$ considers the euclidean projection of the input vector $\mathbf{z}$ onto the probability simplex, which can be defined as the following optimization problem:

$$\begin{aligned} &\min_{\mathbf{p}\in\mathbb{R}^n} \frac{1}{2}\|\mathbf{p}-\mathbf{z}\|^2 \\ &s.t. \mathbf{p}^\top \mathbf{1} = 1, \quad \mathbf{p} \geq 0. \end{aligned} \tag{11}$$

Then, the Lagrangian of the optimization problem in Eq. (11) is:

$$\mathcal{L}(\mathbf{p},\boldsymbol{\alpha},\beta) = \frac{1}{2}\|\mathbf{p}-\mathbf{z}\|^2 - \boldsymbol{\alpha}^\top \mathbf{p} + \beta(\mathbf{1}^\top \mathbf{p} - 1). \tag{12}$$

The optimal $(\mathbf{p}^*, \boldsymbol{\alpha}^*, \beta^*)$ must satisfy the following Karush-Kuhn-Tucker conditions:

$$\mathbf{p}^* - \mathbf{z} - \boldsymbol{\alpha}^* + \beta^* \mathbf{1} = \mathbf{0}, \tag{13}$$

$$\mathbf{1}^\top \mathbf{p}^* = 1, \quad \mathbf{p}^* \geq \mathbf{0}, \quad \boldsymbol{\alpha}^* \geq \mathbf{0}, \tag{14}$$

$$\alpha_i^* p_i^* = 0, \quad \forall i \in \{1,\ldots,n\}. \tag{15}$$

If for $\forall i \in \{1,\ldots,n\}$ we have $p_i^* > 0$, then from Eq. (15) we must satisfy $\alpha_i^* = 0$. Thus, from Eq. (13) we can get $p_i^* = z_i - \beta^*$. Let $S(\mathbf{z}) = \{j \in \{1,\ldots,n\} | p_j^* > 0\}$. From Eq. (14) we obtain $\sum_{j\in S(\mathbf{z})}(z_j - \beta^*) = 1$, which yields the Line 3 in Algorithm 1, i.e., $\beta^* = \tau(\mathbf{z})$. Again from Eq. (15), we have that $\alpha_i^* > 0$ implies $p_i^* = 0$, which from Eq. (13) implies $\alpha_i^* = \beta^* - z_i \geq 0$, i.e., $z_i \leq \beta^*$ for $i \notin S(\mathbf{z})$. That's to say, if the element z_i less than threshold β^*, then α_i^* will larger than 0 and output p_i must be reset to 0 to satisfy Eq. (15). Thus, we can generate the sparse values and have the procedure in Algorithm 1.

3.4 Improving Structure Learning Efficiency and GCN Revisiting

For large scale graphs, it will be computation expensive to calculate the similarities between each pair of nodes during

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

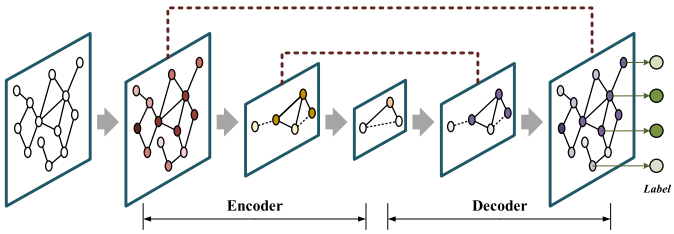


Fig. 1. The architecture of node classification. This simplified example contains two encoder blocks and two decoder blocks, where skip connections are used to fuse information between encoder and decoder. The known node labels are denoted as green circles.

the learning of structure $\mathbf{S}_i^k$. If we take graph's homophily property [70] into account (i.e., similar nodes tends to stick to each other), it is rational for us to constrain the calculation process within the node's h-hop neighborhood ($h = 2$ or 3). Therefore, the computation cost of $\mathbf{S}_i^k$ can be greatly reduced.

After having obtained the refined graph structure $\mathbf{S}_i^k$, we conduct graph convolution and pooling operations in the following layers based on $\tilde{\mathbf{H}}_i^k$ and $\mathbf{S}_i^k$ (instead of $\mathbf{A}_i^k$). Thus, Eq. (1) can be simplified as follows:

$$\mathbf{H}_i^k = \sigma(\mathbf{S}_i^k \tilde{\mathbf{H}}_i^k \mathbf{W}^k). \tag{16}$$

Since the learned $\mathbf{S}_i^k$ satisfies $\sum_{q=1}^{n_i^k} \mathbf{S}_i^k(p,q) = 1$, therefore we have the diagonal matrix $\mathbf{D}_i^k = \mathrm{D}iag(d_1, d_2, \ldots, d_{n_i^k})$ with $d_p = \sum_q^{n_i^k} \mathbf{S}_i^k(p,q)$, which degenerates to identity matrix $\mathbf{I}_i^k$.

3.5 The Overall Architectures

Figs. 1 and 2 provide overviews of our devised two architectures in node classification and graph classification tasks, respectively. The proposed MVPool operator is composed of two major components that are explicitly depicted in Fig. 2: (I) graph pooling, which preserves a subset of important nodes and forms a smaller induced subgraph; and (II) structure learning, which learns a refined graph structure for the pooled subgraph. The advantage of our proposed structure learning mechanism lies in its capability of preserving the underlying graph structure information, which will facilitate the message passing procedure. As in this illustrative example, the pooled subgraph might exist isolated nodes but intuitively ought to be connected, thus it would hinder the information propagation in subsequent layers, especially when aggregating information from its neighborhood nodes. The whole architecture is the stacking of multiple graph convolution and pooling operations, thus making it possible to learn graph representations in a hierarchical way.

For node classification task, we utilize an encoder and decoder architecture inspired by [31], [59]. First, several encoder blocks are stacked to encode the graph information, where each encoder block is made up of a GCN layer [12] and a MVPool layer. Then, the same number of decoder blocks are stacked in the decoding procedure, where each decoder block consists of a up-sampling layer (described in Section 3.6) and a GCN layer [12]. We also utilize residual connection technique [2] to promote representation propagation between the encoder and decoder parts, where the concatenation operation is used in the implementation. At last, we apply an additional GCN layer for predicting the node labels.

For graph classification task, graph level representations are needed. Thus, we use the encoder module to capture the graph information, then a readout function (described in Section 3.6) is utilized to summarize the node representations in each level, and the final graph level representation is the addition of different levels' summarizations. Finally, the graph level representation is fed into a Multi-Layer Perceptron (MLP) with softmax layer to perform graph classification.

3.6 Up-sampling Layer, Readout and Loss Function

In this subsection, we introduce the specific component settings in node classification and graph classification architectures, respectively.

Up-sampling Layer. For node classification in Fig. 1, to restore from the pooled graph to original graph, we preserve the locations of the selected nodes in the pooling procedure and then in the up-sampling layer we use this position information to put the corresponding nodes back to their original positions in the graph, which is formulated as follows:

$$\mathbf{H}_i^{k+1} = distribute(\mathbf{0}_{n_i^{k+1} \times d}, \mathbf{H}_i^k, idx), \tag{17}$$

where idx represents the selected node indices. distribute$(\cdot)$ is the up-sampling operation that distributes representation vectors in $\mathbf{H}_i^k$ into $\mathbf{0}_{n_i^{k+1} \times d}$ where the corresponding values are updated according to the indices in idx.

Readout Function. As for graph classification demonstrated in Fig. 2, the neural network architecture repeats the graph convolution and pooling operations for several times, thus we would observe multiple subgraphs with different size in each level: $\mathbf{H}_i^1, \mathbf{H}_i^2, \ldots, \mathbf{H}_i^K$. To generate a fixed size graph level representation, we devise a readout function that aggregates all the node representations in the subgraph. Here, we simply use the concatenation of mean-

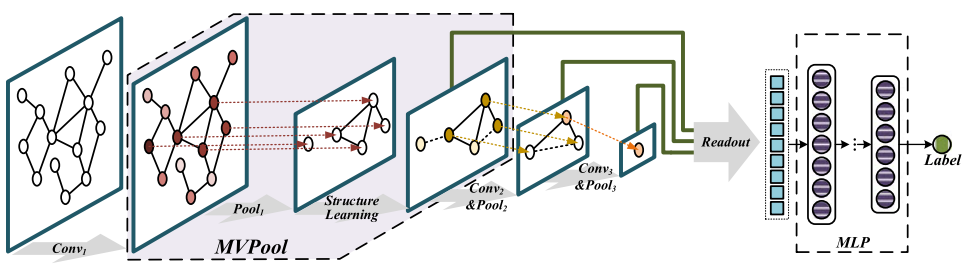


Fig. 2. The architecture of graph classification. The dashed box demonstrates the workflow of MVPool, which involves graph pooling and structure learning. The learned edges are represented as dashed lines in the graph.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Statistics of the Node Classification Datasets

Datasets	$\#\|\mathcal{V}\|$	$\|Attr\|$	$\#\|c\|$	Train	Valid	Test
Cora	2,708	1,433	7	140	500	1,000
Citeseer	3,327	3,703	6	120	500	1,000
Pubmed	19,717	500	3	60	500	1,000
Coauthor-CS	18,333	6,805	15	300	450	17,583
Coauthor-Phy	34,493	8,415	5	100	150	34,243

TABLE 2
Statistics of the Graph Classification Datasets

Datasets	$\#\|\mathcal{G}\|$	$\#\|\mathcal{V}\|$	Avg.$\|\mathcal{V}\|$	Avg.$\|\mathcal{E}\|$	$\#\|c\|$
ENZYMES	600	19,580	32.63	62.14	6
PROTEINS	1,113	43,471	39.06	72.82	2
D&D	1,178	334,925	284.32	715.66	2
NCI1	4,110	122,747	29.87	32.30	2
NCI109	4,127	122,494	29.68	32.13	2
Mutagenicity	4,337	131,488	30.32	30.77	2
IMDB	1,500	19,500	13.00	65.94	3
REDDIT-12K	11,929	4,669,116	391.41	456.89	11

pooling and max-pooling in each subgraph as follows:

$$\mathbf{r}_i^k = \mathcal{R}(\mathbf{H}_i^k) = \sigma\left(\frac{1}{n_i^k}\sum_{p=1}^{n_i^k}\mathbf{H}_i^k(p,:)||\max_{q=1}^{d}\mathbf{H}_i^k(:,q)\right), \tag{18}$$

where $\sigma(\cdot)$ is a nonlinear activation function and $\mathbf{r}_i^k \in \mathbb{R}^{2d}$. We then add the readout outputs of different levels to form our final graph level representation:

$$\mathbf{z}_i = \mathbf{r}_i^1 + \mathbf{r}_i^2 + \cdots + \mathbf{r}_i^K, \tag{19}$$

which summarizes different levels' graph representations.

Loss Function. Finally, we feed the node or graph level representation into MLP layer with softmax classifier, and the loss function is defined as the cross-entropy of predictions over the labels:

$$\hat{\mathbf{Y}} = softmax(\mathrm{MLP}(\mathbf{Z}))\mathcal{L} = -\sum_{i\in L}\sum_{j=1}^{c}\mathbf{Y}_{ij}log\hat{\mathbf{Y}}_{ij}, \tag{20}$$

where $\hat{\mathbf{Y}}_{ij}$ represents the predicted probability that v_i or $\mathcal{G}_i$ belongs to class j, and $\mathbf{Y}_{ij}$ is the ground truth. L denotes the training set of nodes or graphs that have labels.

4 Experiments and Analysis

4.1 Datasets

For node classification and clustering tasks, we utilize three commonly used citation benchmarks[1]: *Cora*, *Citeseer*, *Pubmed* [12], [17], [71] and two widely used coauthor datasets: *Coauthor-CS* and *Coauthor-Physics* [65], whose statistics are summarized in Table 1 with more description as follows. The nodes and edges in citation datasets represent the documents and its citation relationships, respectively. The nodes and edges in coauthor datasets represent the authors and the co-authorships. The attributes of each node is the bag-of-word representations determined by the word dictionary size. We conduct semi-supervised node classification under the transductive setting, i.e., it can have access to all the node's attribute information as well as the whole graph structure but only a part of labeled nodes. We strictly follow the same experimental settings in previous work [12], [17], [71], where 20 nodes per class are used for model training, and 500 nodes are selected for validation, then 1000 nodes are utilized for testing. For fair comparison, we use both standard splits (released in [71]) and random splits to evaluate the performance of different baselines.

For graph classification and clustering tasks, we adopt six widely used public benchmarks[2] for empirical studies. Statistics of the six datasets are summarized in Table 2 with detailed descriptions as follows: *ENZYMES* [72] is a dataset of protein tertiary structures, and each enzyme belongs to one of the 6 EC top-level classes. *PROTEINS* and **D&D** [73] are two protein graph datasets, where nodes represent the amino acids and two nodes are connected by an edge if they are less than 6 Angstroms apart. The label indicates whether or not a protein is a non-enzyme. **NCI1** and **NCI109** [10] are two biological datasets screened for activity against non-small cell lung cancer and ovarian cancer cell lines, where each graph is a chemical compound with nodes and edges representing atoms and chemical bonds, respectively. *Mutagenicity* [9] is a chemical compound dataset of drugs, which can be categorized into two classes: mutagen and non-mutagen. *IMDB* and *REDDIT-12K* [22] are two large-scale social network graphs. We perform graph classification task under inductive setting; that's to say, testing data are not available during the training procedure. Our source code is available at https://github.com/cszhangzhen/MVPool.

4.2 Baselines and Experimental Settings

4.2.1 Node Classification Baselines

We divide the node classification baselines into the following three groups:

Embedding Based Approaches. In this category, we include three embedding based models as baselines. Multi-Layer Perceptron (MLP) projects node feature into latent vector space for node classification. DeepWalk [41] tries to learn node embeddings via exploring graph structure. Planetoid [71] utilizes node feature as well as graph structure information for generating comprehensive node representations.

Semi-Supervised Convolutional Models. Methods in this category mainly focus on performing convolution operation on graphs. Six typical graph neural networks including Chebyshev [7], GCN [12], GAT [17], SGC[47], ARMA[13], APPNP[65] are chosen for comparison.

Hierarchical Representation Algorithms. As for this category, we consider two recently proposed models H-GCN [59] and g-U-Nets [31] as baselines, where the graph coarsening or graph pooling operation is inserted into the graph neural networks to enable hierarchical representation learning.

1. https://github.com/kimiyoung/planetoid/tree/master/data

2. Benchmarks are publicly available at https://ls11-www.cs.tu-dortmund.de/staff/morris/graphkerneldatasets

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

4.2.2 Graph Classification Baselines

The graph classification baselines are categorized into the following three groups:

Graph Kernel Methods. This group of methods perform graph classification by utilizing carefully designed kernels. We choose three classical algorithms: GRAPHLET [74], Shortest-Path Kernel (SP) [75] and Weisfeiler-Lehman Kernel (WL) [10] as baselines.

Graph Neural Networks. Approaches in this group include representative graph neural networks: GCN [12], GraphSAGE [15] and GAT [17], which are designed to learn meaningful node level representations. Therefore, we employ our proposed readout function Eq. (18) to summarize the node representations for graph classification.

Graph Pooling Models. In this group, we consider numerous models that combine GNNs with pooling operator for graph level representation learning. Set2Set [53] and DGCNN [55] are two novel global graph pooling algorithms. Another seven hierarchical graph pooling models including DiffPool [22], gPool [31], SAGPool [35], EdgePool [32], EigenPool [34], iPool [52] and HGP-SL [37] are also compared as baselines.

4.2.3 MVPool Variants.

To further analyze the effectiveness of our proposed model, we consider four variants here: MVPool which discards the structure learning layer to verify the effectiveness of our proposed structure learning module, MVPool_{HOP} which removes the structure learning layer and connects all the nodes within its h-hops, $\mathrm{MVPool\text{-}SL}_{DEN}$ (**DEN**se) which employs the structure learning layer to learn a dense graph structure with softmax function defined in Eq. (9) and MVPool-SL which utilizes sparsemax function define in Eq. (10) to learn a sparse graph structure. Both $\mathrm{MVPool\text{-}SL}_{DEN}$ and MVPool-SL use efficiency improved structure learning strategy discussed in Section 3.4.

4.2.4 Experiment and Parameter Settings.

For transductive node classification task, in standard splits, we use the same training, validation and testing splittings released by previous work [71], which are also used in [12], [17]. Thus, we directly cite the baselines' results reported in Pytorch Geometric benchmark [76] and their original papers; in random splits, we randomly choose a same number of training, validation and testing samples as standard splittings. All these procedures are repeated 10 times and then we report the average performance with standard derivation. For model's implementation, dropout mechanism [77] with drop rate setting as 0.92 is employed to the node feature matrix. We search the learning rate and weight decay in the range of $\{0.1, 0.01, 0.001, 0.0001\}$, pooling ratio $r \in (0.0, 0.9]$, and encoder layers $K \in [1, 5]$, respectively. The node representation dimension in Cora and Citeseer is fixed as 64, and in Pubmed dataset we set its dimension as 16. For model's training, we stop the optimization procedure when the iteration reaches 200 epochs.

For inductive graph classification task, we randomly split each dataset into three parts by following [34]: 80 percent as training set, 10 percent as validation set and the remaining 10 percent as test set. We repeat this randomly splitting process 10 times, and the average performance with standard derivation is reported. For baseline models, we use the source code released by the authors, and their hyper-parameters are tuned to be optimal based on the validation set. In order to ensure a fair comparison, the same neural network architectures are used for baselines and our proposed model. The dimension of node representations is set as 128 for all methods and datasets in graph classification task. We implement our proposed MVPool with PyTorch Geometric[3] [76], and the Adam optimizer [78] is utilized to optimize the model's parameters. The learning rate and weight decays are searched in $\{0.1, 0.01, 0.001, 1e^{-4}, 1e^{-5}\}$, pooling ratio $r \in (0.0, 0.9]$ and layers $K \in [1, 5]$. The MLP with dropout schema [77] consists of three fully connected layers with number of neurons in each layer setting as 256, 128, 64 followed by a softmax classifier. The max iteration is defined as 1,000 and early stopping criterion is employed in the training process, i.e., we stop training if the validation loss dose not decrease for 100 consecutive epochs.

In node and graph level clustering tasks, we utilize the encoder part to obtain its representations, which are then fed into a traditional clustering algorithm. For simplicity, we adopt K-means to conduct clustering, and the number of clusters (e.g., K) is set as the number of classes in datasets. Since K-means is sensitive to the initialized cluster centers, we repeat this process for 10 times and report the average results. NMI [79] and ARI [80] are employed to evaluate the clustering results' quality.

4.3 Performance on Node Classification

The node classification performance in terms of accuracy is summarized in Tables 3 and 4. First, we can observe that our proposed MVPool-SL achieves state-of-the-art performance across all datasets. For embedding based baselines, MLP uses only node feature information and the rich structure information in graph data is ignored, thus leading to unsatisfied performance. DeepWalk is a representative graph embedding method, however it performs worse in Citeseer and Pubmed datasets. This is because it explores graph structure information only, and the two-stage formulation also makes it impossible to be trained in an end-to-end manner. Planetoid integrates both node feature and graph structure information and achieves better performance than the other two methods, which indicates the importance of taking both of them into consideration.

Instead of capturing the graph structure information via random walks like DeepWalk [41] and Planetoid [71], graph convolutional networks incorporate the relations between different neighbors in a more flexible way. Through iteratively aggregating information from nodes and its neighborhood, GNN models encode node feature and graph structure into unified representation space and achieve promising results. Among them, GAT utilizes attention mechanism to aggregate information and outperforms other baselines in this group. Meanwhile, it's worth noting that hierarchical representation learning algorithms H-GCN and g-U-Nets surpass vanilla GCN with different gains. The difference between H-GCN (or g-U-Nets) and GCN is the use

3. https://pytorch-geometric.readthedocs.io/en/latest/

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Node Classification in Terms of Accuracy With Standard Deviation

Models	Cora	Citeseer	Pubmed
Standard Splits			
MLP	55.1%	46.5%	71.4%
DeepWalk [41]	67.2%	43.2%	65.3%
Planetoid [71]	75.7%	64.7%	77.2%
Chebyshev [7]	81.2%	69.8%	74.4%
GCN [12]	81.5%	70.3%	79.0%
GAT [17]	$83.0 \pm 0.7\%$	$72.5 \pm 0.7\%$	$79.0 \pm 0.3\%$
SGC[47]	$81.7 \pm 0.1\%$	$71.3 \pm 0.2\%$	$78.9 \pm 0.1\%$
ARMA[13]	$82.8 \pm 0.6\%$	$72.3 \pm 1.1\%$	$78.8 \pm 0.3\%$
APPNP[65]	$83.3 \pm 0.5\%$	$71.8 \pm 0.5\%$	$80.1 \pm 0.2\%$
H-GCN [59]	$84.5 \pm 0.5\%$	$72.8 \pm 0.5\%$	$79.8 \pm 0.4\%$
g-U-Nets [31]	$84.4 \pm 0.6\%$	$73.2 \pm 0.5\%$	$79.6 \pm 0.2\%$
MVPool	$82.8 \pm 0.4\%$	$69.9 \pm 1.5\%$	$77.2 \pm 0.4\%$
MVPool_{HOP}	$83.3 \pm 0.5\%$	$71.7 \pm 0.7\%$	$78.2 \pm 0.2\%$
MVPool-SL_{DEN}	$83.1 \pm 0.5\%$	$70.7 \pm 1.3\%$	$79.3 \pm 0.4\%$
MVPool-SL	$\mathbf{85.0 \pm 0.4\%}$	$\mathbf{73.5 \pm 0.5\%}$	$\mathbf{80.5 \pm 0.4\%}$
Random Splits			
Chebyshev [7]	$77.8 \pm 2.2\%$	$67.7 \pm 1.7\%$	$75.8 \pm 2.2\%$
GCN [12]	$79.4 \pm 1.9\%$	$68.1 \pm 1.7\%$	$77.4 \pm 2.4\%$
GAT [17]	$81.0 \pm 1.4\%$	$69.2 \pm 1.9\%$	$78.3 \pm 2.3\%$
SGC[47]	$80.2 \pm 1.6\%$	$68.7 \pm 1.6\%$	$76.5 \pm 2.4\%$
ARMA[13]	$80.7 \pm 1.4\%$	$68.9 \pm 1.6\%$	$77.7 \pm 2.6\%$
APPNP[65]	$82.2 \pm 1.5\%$	$70.0 \pm 1.4\%$	$79.4 \pm 2.2\%$
H-GCN [59]	$81.1 \pm 1.2\%$	$69.5 \pm 1.4\%$	$77.8 \pm 1.6\%$
g-U-Nets [31]	$83.1 \pm 1.6\%$	$70.5 \pm 2.1\%$	$78.7 \pm 2.9\%$
MVPool	$82.8 \pm 1.0\%$	$68.5 \pm 1.8\%$	$77.4 \pm 2.4\%$
MVPool_{HOP}	$82.4 \pm 1.0\%$	$68.9 \pm 1.7\%$	$78.3 \pm 1.8\%$
MVPool-SL_{DEN}	$83.0 \pm 0.8\%$	$69.2 \pm 1.5\%$	$79.0 \pm 2.0\%$
MVPool-SL	$\mathbf{83.6 \pm 1.2\%}$	$\mathbf{72.6 \pm 1.1\%}$	$\mathbf{80.0 \pm 1.8\%}$

*We use **bold** to highlight wins.*

TABLE 4
Node Classification in Terms of Accuracy With Standard Deviation

Models	Coauthor-CS	Coauthor-Physics
LogReg	86.4±0.9%	86.7±1.5%
MLP	88.3±0.7%	88.9±1.1%
LabelProp	73.6±3.9%	86.6±2.0%
Chebyshev	89.7±0.2%	90.8±1.8%
GCN	91.1±0.5%	92.8±1.0%
GAT	90.5±0.6%	92.5±0.9%
SGC	91.5±0.5%	92.6±1.3%
ARMA	89.7±0.4%	92.6±0.9%
APPNP	89.6±0.4%	90.3±0.2%
H-GCN	88.2±0.3%	91.7±0.4%
g-U-Nets	90.9±0.6%	93.2±1.0%
MVPool-SL	**92.0±0.5%**	**94.0±0.6%**

of pooling layers and the encoder-decoder architecture, which verifies the effectiveness of learning representation in a hierarchical way.

In particular, our proposed MVPool-SL beats H-GCN and g-U-Nets in all datasets. We attribute the performance lift to the devised novel pooling operator. H-GCN utilizes some predefined graph coarsening criterions to scale-down the graph, which has limited capabilities since the supervision information can not guide the coarsening process. On the contrary, g-U-Nets employs a flexible Top-K pooling scheme, however the pooling operation is determined by node feature information only and the induced subgraph might produce lots of isolated nodes, which hinders the information propagation in next layer. As for our proposed MVPool-SL, the node selection is based on the collaboration of different views within different contextual graph information, which could generate more robust node ranking results. Moreover, the structure learning mechanism makes it possible to automatically rebuild the connections between the isolated nodes and the graphs.

Note that the comparisons between MVPool-SL variants also demonstrate the effectiveness of using structure learning layers. MVPool-SL and MVPool-SL_{DEN} obtain better performance than MVPool and MVPool_{HOP}, which justifies the effectiveness of our proposed structure learning layer. Moreover, MVPool_{HOP} performs worse than MVPool-SL. This is because the disconnected nodes are still unreachable within its h-hops. MVPool-SL further outperforms MVPool-SL_{DEN}, which indicates the learned dense graph structure might introduce additional noisy information and degenerate the performance. Furthermore, in the real-world scenario, graphs usually have sparse topologies, thus our proposed MVPool-SL could learn more reasonable graph structures compared with MVPool-SL_{DEN}.

At last, we observe that the state-of-the-art GNN baselines perform dramatically different on the standard splits and random splits. As we can see in Table 3, the improved performance among recent proposed approaches is somewhat limited on the standard splits. Their performance of multiple runs is relatively stable since the standard derivation is very small. However, when testing existing models on random splits, their results decrease with different scales and their standard derivations increase significantly compared with standard splits. This indicates that current GNN approaches are sensitive to the data splits and might have overfitted on the fixed, standard splits, thus having poor generalization capabilities. Nonetheless, our proposed MVPool-SL outperforms all the baselines under both standard splits and random splits. More specifically, MVPool-SL achieves larger gains compared with H-GCN and g-U-Nets in random splits, and it shows better generalization capabilities, because the random splits have little effect on its performance.

4.4 Performance on Graph Classification

The graph classification performance is reported in Table 5. To summarize, we have the following observations. First of all, a general observation we can draw from the results is that our proposed MVPool-SL consistently outperforms other state-of-the-art baselines among all datasets. For instance, our method achieves about 0.8 percent improvement over the best baseline in PROTEINS dataset, which is 14.02 percent relative improvement over GCN with no hierarchical pooling mechanism. This verifies the necessity of adding graph pooling module.

It is worth noting that the traditional graph kernel based methods demonstrate competitive performance. However, the carefully designed graph kernels typically involve

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 5
Graph Classification in Terms of Accuracy With Standard Deviation (in Percentage)

Baselines	ENZYMES	PROTEINS	D&D	NCI1	NCI109	Mutagenicity	IMDB	REDDIT-12K
GRAPHLET	29.2 ± 5.6	72.2 ± 4.5	72.5 ± 3.8	62.5 ± 2.1	60.9 ± 2.4	56.6 ± 1.7	37.6 ± 1.8	20.7 ± 0.7
SP	42.6 ± 5.3	75.7 ± 2.7	78.7 ± 3.9	67.4 ± 2.8	67.7 ± 2.3	71.6 ± 2.2	48.6 ± 2.6	36.3 ± 0.9
WL	51.2 ± 6.2	76.2 ± 4.0	76.4 ± 2.4	76.7 ± 2.0	76.2 ± 2.5	80.3 ± 1.7	46.7 ± 3.6	38.1 ± 1.0
GCN	43.7 ± 3.4	75.2 ± 3.6	73.3 ± 4.5	76.3 ± 1.8	75.9 ± 1.8	79.8 ± 1.6	46.6 ± 1.2	43.4 ± 1.9
GraphSAGE	38.0 ± 3.7	74.0 ± 4.3	75.8 ± 3.9	74.7 ± 1.3	74.2 ± 2.9	78.8 ± 1.2	46.6 ± 2.0	39.1 ± 0.6
GAT	39.8 ± 3.7	74.7 ± 4.0	77.3 ± 3.7	74.9 ± 1.7	75.8 ± 2.7	78.9 ± 2.1	50.1 ± 1.5	42.3 ± 0.6
Set2Set	33.2 ± 3.2	79.3 ± 0.8	70.8 ± 0.8	69.6 ± 1.3	73.7 ± 1.7	80.8 ± 0.7	47.8 ± 1.0	43.4 ± 0.9
DGCNN	32.2 ± 3.9	80.0 ± 0.4	70.1 ± 1.2	74.1 ± 2.2	78.2 ± 1.3	80.4 ± 1.0	50.6 ± 1.3	44.5 ± 0.8
DiffPool	60.6 ± 3.9	79.9 ± 3.0	78.6 ± 1.3	77.7 ± 0.8	77.1 ± 1.5	80.8 ± 1.1	49.1 ± 1.8	45.5 ± 0.8
EigenPool	64.0 ± 2.5	78.8 ± 1.1	78.6 ± 1.4	77.2 ± 1.0	76.0 ± 1.4	80.1 ± 0.7	49.5 ± 1.5	45.8 ± 0.5
gPool	43.3 ± 2.9	80.7 ± 1.8	77.0 ± 1.3	76.3 ± 1.4	76.6 ± 1.4	80.3 ± 1.5	48.9 ± 1.7	45.3 ± 0.7
SAGPool	44.0 ± 4.2	81.7 ± 2.2	78.7 ± 2.3	77.9 ± 1.6	75.7 ± 1.5	79.7 ± 0.8	48.9 ± 1.2	42.7 ± 1.7
iPool	63.4 ± 3.6	83.0 ± 1.8	77.1 ± 2.1	76.4 ± 1.9	78.8 ± 2.0	80.5 ± 0.7	48.3 ± 2.0	45.2 ± 1.4
EdgePool	65.3 ± 4.4	82.4 ± 0.8	79.2 ± 2.6	76.6 ± 1.0	79.0 ± 1.9	81.4 ± 0.9	48.4 ± 1.0	44.1 ± 1.0
HGP-SL	68.8 ± 2.1	84.9 ± 1.6	81.0 ± 1.3	78.5 ± 0.8	80.7 ± 1.2	82.2 ± 0.6	49.6 ± 1.2	44.4 ± 1.2
MVPool	62.4 ± 2.5	82.2 ± 1.2	78.4 ± 1.5	77.5 ± 1.3	77.2 ± 1.1	80.2 ± 0.8	48.6 ± 1.0	44.0 ± 1.5
MVPool_{HOP}	63.6 ± 2.2	83.2 ± 1.7	79.3 ± 1.3	77.9 ± 1.0	78.3 ± 1.4	80.9 ± 1.0	49.3 ± 0.8	44.5 ± 0.9
MVPool-SL_{DEN}	64.6 ± 2.3	83.1 ± 1.1	79.6 ± 1.4	78.7 ± 1.1	79.1 ± 1.3	81.7 ± 1.2	50.0 ± 1.2	46.2 ± 1.1
MVPool-SL	$\mathbf{69.5 \pm 2.4}$	$\mathbf{85.7 \pm 1.2}$	$\mathbf{82.2 \pm 1.4}$	$\mathbf{80.1 \pm 1.3}$	$\mathbf{81.9 \pm 1.6}$	$\mathbf{83.7 \pm 0.7}$	$\mathbf{52.0 \pm 0.8}$	$\mathbf{47.4 \pm 1.3}$

*We use **bold** to highlight wins.*

massive human domain knowledge, which has difficulty in generalizing to graphs with arbitrary structures. Furthermore, the two-stage procedure of extracting graph features and performing graph classification might result in sub-optimal performance.

Being consistent with previous work's findings [34], we also observe that the GNNs group can not achieve satisfied results. We argue that the major reason is because they ignore the graph structure information when globally summarizing the node representations, which further verifies the necessity of adding graph pooling module. In particular, the global pooling approaches Set2Set and DGCNN are surpassed by most of the hierarchical pooling methods with a few exceptions. This is because their learned graph representations are still "flat", and the hierarchical structure information or functional units in the graph are ignored, which play an important role in predicting the entire graph labels.

We note that the hierarchical pooling models can achieve relative better performance among most baselines, which further shows the effectiveness of the hierarchical pooling mechanism. Among them, gPool and SAGPool perform poorly in ENZYMES dataset. This can be attributed to the limited training samples per class resulting in the neural network overfitting. EdgePool gains relatively better performance in this group of competitors, which scales down the size of graphs by contracting each pair of nodes in the graph and always has the pooling ratio as 0.5. This poor pooling flexibility restricts its capability, e.g., performing poor in NCI1 dataset. We would like to talk about the advantages of MVPool-SL over our previous work HGP-SL [37]. First, HGP-SL uses only local graph information to evaluate node importance and loses the global perspective, while MVPool utilizes local as well as global graph information, i.e., the calculating and propagating of PageRank values. Second, the collaboration of different views make it possible to neglect the noisy information and generate more robust node rankings. Obviously, our proposed MVPool-SL outperforms HGP-SL with different gains for all settings, which shows the benefit of collaboration between multiple views. In the end, similarly to semi-supervised node classification task, the comparisons between different MVPool-SL variants in inductive graph classification task reaches same conclusion, which has been analyzed in Section 4.3. Thus, we omit it in this subsection due to the limited space.

4.5 Performance on Clustering

For node clustering task, we first utilize graph embedding or GNN models to obtain the corresponding node representations and then perform Kmeans based on the learned node representations. Table 6 shows the performance of node clustering task. The results indicate that MVPool-SL

TABLE 6
Node Clustering in Terms of NMI and ARI in Percentage

Datasets	Cora		Citeseer		Pubmed	
Evaluations	NMI	ARI	NMI	ARI	NMI	ARI
Kmeans	17.2	10.2	20.7	16.7	31.4	28.1
DeepWalk	42.4	29.8	23.2	9.20	14.2	7.12
Chebyshev	53.5	41.7	34.2	30.1	34.4	37.2
GCN	55.8	54.8	35.4	36.3	35.9	39.7
GAT	56.1	47.5	36.6	32.6	37.3	38.2
SGC	56.6	48.9	36.9	33.4	35.1	32.7
ARMA	58.1	55.7	36.2	33.6	34.5	37.5
APPNP	58.4	53.5	37.9	35.8	37.2	39.0
H-GCN	52.3	45.8	34.7	27.2	31.4	27.7
g-U-Nets	58.9	54.1	37.4	37.1	37.7	39.1
MVPool-SL	**61.2**	**56.4**	**40.6**	**39.2**	**38.5**	**42.7**

*We use **bold** to highlight wins.*

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

TABLE 7
Graph Clustering in Terms of NMI and ARI in Percentage

Datasets	PROTEINS		NCI109		Mutagenicity	
Evaluations	NMI	ARI	NMI	ARI	NMI	ARI
GCN	15.6	22.2	30.2	38.6	38.1	48.2
GraphSAGE	13.9	20.1	21.2	27.6	31.2	39.1
GAT	17.1	24.0	33.5	41.8	25.2	30.2
Set2Set	16.6	21.0	21.8	28.6	32.6	37.5
DGCNN	16.3	22.6	18.8	23.3	26.5	24.1
DiffPool	16.9	23.9	34.8	42.1	35.4	44.7
EigenPool	16.1	22.3	28.0	36.1	39.6	49.8
gPool	16.5	21.2	17.2	18.4	26.5	33.7
SAGPool	16.9	21.5	18.4	23.6	26.2	33.9
iPool	16.8	21.6	31.3	40.1	36.8	46.3
EdgePool	18.0	22.5	26.1	28.9	39.4	49.8
HGP-SL	17.7	23.5	30.4	39.0	36.5	46.4
MVPool-SL	**20.8**	**27.3**	**36.7**	**44.2**	**42.5**	**52.6**

*We use **bold** to highlight wins.*

has achieved the best performance on both two metrics compared with all the baselines. As we can see from the Table 6, Kmeans and DeepWalk perform dramatically different on different datasets. For instance, Kmeans achieves relatively worse performance on Cora and Citeseer datasets, but performs well on Pubmed dataset compared with DeepWalk. This is because Kmeans utilizes node attributes information only, while DeepWalk explores graph structure information only. According to the data statistics in Table 1, Cora and Citeseer have very high attribute dimension compared with Pubmed, thus it may contain lots of noisy information that impairs the clustering performance. While in Pubmed, the quality of node attributes is relatively high (e.g., 500 elaborately designed features), therefore Kmeans outperforms DeepWalk with a large scale of gains.

By integrating both node attributes and graph structure information, existing GNN baselines beat attribute-based Kmeans as well as structure-based DeepWalk with different gains. However, we did not observe a clear winner among different GNN baselines except our proposed MVPool-SL. We attribute the improved performance to our designed novel graph pooling operator. Through incorporating pooling mechanism, it's possible for MVPool-SL to explore large neighborhood information and learn hierarchical node representations, which is important for clustering task. Moreover, the collaboration of multiple views has the benefit of fully utilizing various contextual information in the graph.

We also report the graph clustering performance in Table 7. In summary, our proposed MVPool-SL beats all the baselines among all the datasets. Similarly to node clustering task, there is no clear winner among different compared graph pooling approaches. For example, EdgePool performs quite well on PROTEINS and Mutagenicity datasets, however it can not achieve satisfied results on NCI109 datasets. The underlying reason is because the graph structure in different datasets is significantly different (e.g., the average number of edges in PROTEINS is twice as large as NCI109), which might have a great impact on the clustering task. In contrast, MVPool-SL consistently obtains promising performance across different datasets. The performance lift can be attributed to the well-designed multi-view learning mechanism, thus the combination of different views can produce relatively robust results.

TABLE 8
MVPool-SL Performance With Different Architectures

Architectures	PROTEINS	NCI109	Mutagenicity
MVPool-SL$_{GCN}$	85.71±1.24	81.86±1.58	83.65±0.71
MVPool-SL$_{GAT}$	86.02±1.13	80.54±1.45	82.92±0.52
MVPool-SL$_{SAGE}$	85.29±1.22	80.78±1.16	82.48±0.74
gPool-SL	81.25±1.27	77.71±1.22	80.42±1.08
SAGPool-SL	82.67±1.42	78.01±1.50	80.00±1.22

4.6 Ablation Study and Visualization

4.6.1 Graph Convolutional Neural Network Architectures

As mentioned in previous sections, our proposed MVPool-SL can be integrated into various graph neural network architectures. We consider three most widely used graph convolutional architectures as our model's building block to investigate the affect of different convolution operations: GCN [12], GraphSAGE [15] and GAT [17]. We evaluate them on three datasets, which cover both small and large datasets. Their results are shown in Table 8. Similar results can also be found in the remaining datasets as well as transductive node classification datasets. As demonstrated in Table 8, the performance on graph classification varies depending on which dataset and the type of GNN in MVPool-SL are chosen. In addition, we also combine the top-K selection procedure proposed in gPool and SAGPool with our proposed structure learning. We name them as gPool-SL and SAGPool-SL for short. From the results, we observe that gPool-SL and SAGPool-SL outperform gPool and SAGPool by incorporating the structure learning mechanism, which verifies the effectiveness of our proposed structure learning.

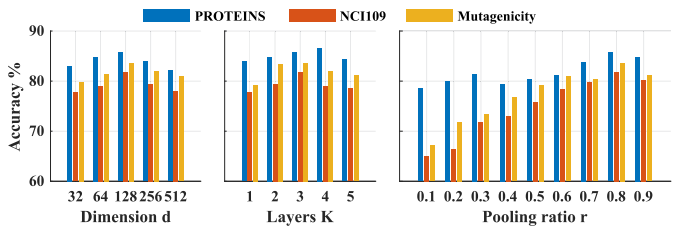


Fig. 3. Hyper-parameter sensitivity analysis on graph classification task.

TABLE 9
MVPool-SL Performance With Different Individual Views

Views	Cora	Citeseer	Pubmed
MVPool-SL$_{S}$	83.1±0.5%	70.9±0.8%	77.9±0.4%
MVPool-SL$_{F}$	83.3±0.5%	71.1±0.7%	78.2±0.2%
MVPool-SL$_{SF}$	83.9±0.6%	72.0±0.5%	79.6±0.4%
MVPool-SL	85.0±0.4%	73.5±0.5%	80.5±0.4%

S, F and SF represent Structure, Feature and Structure-Feature specific views.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

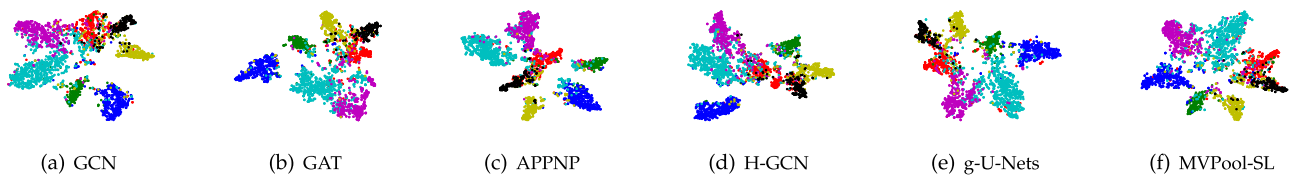


Fig. 4. Visualization of node embeddings generated by different graph neural network methods. Different colors represent different classes. As we can see from the figure, GCN and GAT can not clearly separate different classes from each other.

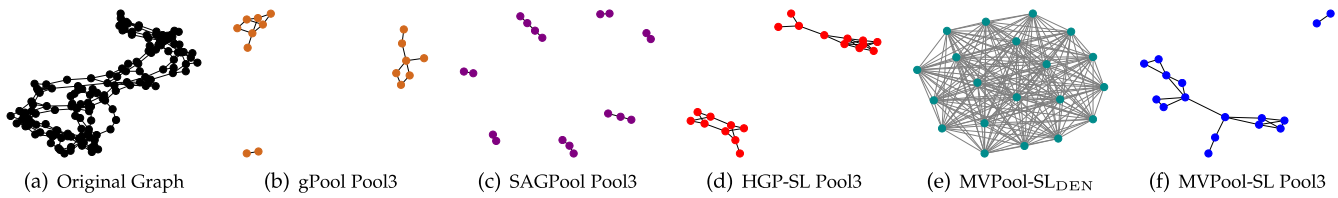


Fig. 5. Visualization of different graph pooling methods. As we can see from the figure, gPool and SAGPool lost the underlying graph structure information during the pooling procedure.

4.6.2 Hyper-parameter Analysis

We further study the sensitivities of several key hyper-parameters by varying them in different scales. Specifically, we investigate how the number of neural network layers K, graph representation dimension d and pooling ratio r will affect the graph classification performance. As we can see in Fig. 3, MVPool-SL almost achieves the best performance across different datasets when setting $K = 3$, $d = 128$ and $r = 0.8$, respectively. The pooling ratio r cannot be too small, otherwise most of the graph structure information will be lost during the pooling process.

4.6.3 View-specific Pooling Analysis

To analyze the impact of specific views, we show the semi-supervised node classification results under individual views in Table 9. As expected, using individual views separately can not obtain satisfied performance due to the biased node evaluations. We also notice that incorporating both node feature and graph structure information could gain better results. This is because both of them contain rich semantic information in graph-structured data. Furthermore, MVPool-SL utilizes attention mechanism to assign different weights to different views and then promotes the collaboration of different views, which can fully explore the available information in graph and give robust node importance evaluations.

4.6.4 Visualization

Visualization is an additional way to evaluate the learned node embeddings. Due to space limitation, we only visualize the Cora dataset, which consists of 2,708 papers from 7 research fields. Here, we utilize t-SNE [81] to project the paper's embedding vectors into 2-D space. Fig. 4 shows the scatter plots generated by different graph neural network approaches, where the node's color indicates the paper's class. Obviously, baselines like GCN, GAT and APPNP do not achieve satisfactory visualizing results, because there is no clear margin between different classes. In contrast, our proposed MVPool-SL can separate different classes from each other and form relatively compact and homogeneous clusters.

We also utilize networkx[4] to visualize the pooling results of MVPool-SL and several recent baselines. In detail, we randomly sample a graph from PROTEINS dataset, which contains 154 nodes. We build a three layer graph neural network with pooling ratio setting as 0.5, which then generates three pooled graphs with nodes as 77, 39 and 20 respectively. We plot the 3rd pooled graph in Fig. 5. It shows gPool and SAGPool fail to preserve meaningful graph topologies, while MVPool-SL is able to preserve relatively reasonable topology of the original protein graph after pooling.

5 Conclusion

In this paper, we propose a novel graph pooling operator MVPool, which empowers GNNs to learn hierarchical graph representations. It can also be conveniently integrated into various GNN architectures. Specifically, the graph pooling operation adaptively selects a subset of nodes, which makes full use of both node features and graph structure information to evaluate the importance of nodes and perform down-sampling on graphs. The proposed collaboration of multiple views under different graph contextual information could lead to more robust node evaluations. Then, a structure learning layer is stacked on the pooling operation, which aims to learn a refined graph structure that can best preserve the underlying topological information. We combine the proposed MVPool operator with graph convolutional neural networks to conduct both transductive node classification and inductive graph classification as well as clustering tasks. The comprehensive experiments on thirteen widely used benchmarks demonstrate its superiority to a range of state-of-the-art methods.

Acknowledgments

This work was supported in part by the National Key Research and Development Program under Grant 2018YFB1403202, in part by the National Natural Science Foundation of China under Grant 61972349, and in part by

4. https://networkx.github.io/

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

the Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Soft Science Research Project of Zhejiang Province Science and Technology Department under Grant 2020C25035.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognition.*, 2016, pp. 770–778.

[3] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2015, *arXiv:1409.0473*.

[5] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei, "Large-scale video classification with convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 1725–1732.

[6] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 568–576.

[7] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 3844–3852.

[8] P. Yanardag and S. Vishwanathan, "Deep graph kernels," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 1365–1374.

[9] J. Kazius, R. McGuire, and R. Bursi, "Derivation and validation of toxicophores for mutagenicity prediction," *J. Med. Chem.*, vol. 48, no. 1, pp. 312–320, 2005.

[10] N. Shervashidze, P. Schweitzer, E. J. v. Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-Lehman graph kernels," *J. Mach. Learn. Res.*, vol. 12, no. Sep, pp. 2539–2561, 2011.

[11] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203*.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017, *arXiv:1609.02907*.

[13] F. M. Bianchi, D. Grattarola, C. Alippi, and L. Livi, "Graph neural networks with convolutional ARMA filters," 2019, *arXiv: 1901.01343*.

[14] K. Zhan and C. Niu, "Mutual teaching for graph convolutional networks," *Future Gener. Comput. Syst.*, vol. 115, pp. 837–843, 2021.

[15] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.

[16] F. Monti, D. Boscaini, J. Masci, E. Rodola, J. Svoboda, and M. M. Bronstein, "Geometric deep learning on graphs and manifolds using mixture model CNNS," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5115–5124.

[17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2018, *arXiv: 1710.10903*.

[18] C. Morris *et al.*, "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.

[19] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-I. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," *Proc. 35th Int. Conf. Mach. Learn., 2018*, pp. 5453–5462.

[20] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1263–1272.

[21] T. Derr, Y. Ma, and J. Tang, "Signed graph convolutional networks," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 929–934.

[22] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 4800–4810.

[23] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," 2018, *arXiv: 1810.00826*.

[24] F. Errica, M. Podda, D. Bacciu, and A. Micheli, "A fair comparison of graph neural networks for graph classification," 2019, *arXiv: 1912.09893*.

[25] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.*, 2018, pp. 593–607.

[26] Z. Zhang *et al.*, "ANRL: Attributed network representation learning via deep neural networks," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3155–3161.

[27] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 974–983.

[28] W. Fan *et al.*, "Graph neural networks for social recommendation," in *Proc. World Wide Web Conf.*, 2019, pp. 417–426.

[29] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. Thirty-Second AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.

[30] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," 2020, *arXiv: 2003.00982*.

[31] H. Gao and S. Ji, "Graph U-NETS," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 2083–2092.

[32] F. Diehl, "Edge contraction pooling for graph neural networks," 2019, *arXiv: 1905.10990*.

[33] H. Gao, Y. Chen, and S. Ji, "Learning graph pooling and hybrid convolutional operations for text representations," in *Proc. World Wide Web Conf.*, 2019, pp. 2743–2749.

[34] Y. Ma, S. Wang, C. C. Aggarwal, and J. Tang, "Graph convolutional networks with Eigenpooling," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019.

[35] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3734–3743.

[36] F. M. Bianchi, D. Grattarola, and C. Alippi, "Spectral clustering with graph neural networks for graph pooling," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 874–883.

[37] Z. Zhang *et al.*, "Hierarchical graph pooling with structure learning," 2019, *arXiv:1911.05954*.

[38] M. Qu, J. Tang, J. Shang, X. Ren, M. Zhang, and J. Han, "An attention-based collaboration framework for multi-view network representation learning," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2017, pp. 1767–1776.

[39] H. Chen, H. Yin, T. Chen, Q. V. H. Nguyen, W.-C. Peng, and X. Li, "Exploiting centrality information with graph convolutions for network representation learning," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 590–601.

[40] A. Martins and R. Astudillo, "From softmax to sparsemax: A sparse model of attention and multi-label classification," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1614–1623.

[41] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.

[42] A. Grover and J. Leskovec, "NODE2VEC: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.

[43] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.

[44] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.

[45] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. 26th Int. Conf. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.

[46] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, and J. Tang, "Network embedding as matrix factorization: Unifying deepwalk, line, PTE, and NODE2VEC," in *Proc. Int. Conf. Web Search Data Mining*, 2018, pp. 459–467.

[47] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.

[48] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, and M. Sun, "Graph neural networks: A review of methods and applications," 2018, *arXiv: 1812.08434*.

[49] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," 2018, *arXiv: 1812.04202*.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.

[50] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," 2019, *arXiv: 1901.00596*.
[51] J. Huang, Z. Li, N. Li, S. Liu, and G. Li, "Attpool: Towards hierarchical feature representation in graph convolutional networks via attention mechanism," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 6480–6489.
[52] X. Gao, H. Xiong, and P. Frossard, "IPOOL–information-based pooling in hierarchical graph neural networks," 2019, *arXiv: 1907.00832*.
[53] O. Vinyals, S. Bengio, and M. Kudlur, "Order matters: Sequence to sequence for sets," 2015, *arXiv:1511.06391*.
[54] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[55] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4445.
[56] H. Yuan and S. Ji, "Structpool: Structured graph pooling via conditional random fields," in *Proc. Int. Conf. Learn. Representations*, 2020.
[57] S. Rhee, S. Seo, and S. Kim, "Hybrid approach of relation network and localized graph convolutional filtering for breast cancer subtype classification," 2017, *arXiv: 1711.05859*.
[58] I. S. Dhillon, Y. Guan, and B. Kulis, "Weighted graph cuts without eigenvectors a multilevel approach," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 11, pp. 1944–1957, Nov. 2007.
[59] F. Hu, Y. Zhu, S. Wu, L. Wang, and T. Tan, "Hierarchical graph convolutional networks for semi-supervised node classification," 2019, *arXiv: 1902.06667*.
[60] R. Zafarani, M. A. Abbasi, and H. Liu, *Social Media Mining: An Introduction*. Cambridge, U.K.: Cambridge Univ. Press, 2014.
[61] L. C. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.
[62] C. F. Negre *et al.*, "Eigenvector centrality for characterization of protein allosteric pathways," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 52, pp. E12201–E12208, 2018.
[63] L. Page, S. Brin, R. Motwani, and T. Winograd, "The Pagerank citation ranking: Bringing order to the web," Stanford InfoLab, Stanford, CA, USA, Tech. Rep. 1999–66, 1999.
[64] T. H. Haveliwala, "Topic-sensitive Pagerank: A context-sensitive ranking algorithm for web search," *IEEE Trans. Knowl. Data Eng.*, vol. 15, no. 4, pp. 784–796, Jul./Aug. 2003.
[65] J. Klicpera, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," 2018, *arXiv: 1810.05997*.
[66] R. Li, S. Wang, F. Zhu, and J. Huang, "Adaptive graph convolutional neural networks," in *Proc. Thirty-Second AAAI Conf. Artif. Intell.*, 2018, pp. 3546–3553.
[67] B. Jiang, Z. Zhang, D. Lin, J. Tang, and B. Luo, "Semi-supervised learning with graph learning-convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11313–11320.
[68] V. Niculae and M. Blondel, "A regularized framework for sparse and structured neural attention," 2017, *arXiv: 1705.07704*.
[69] W. Wang and M. A. Carreira-Perpin án, "Projection onto the probability simplex: An efficient algorithm with a simple proof, and an application," 2013, *arXiv:1309.1541*.
[70] M. McPherson, L. Smith-Lovin, and J. M. Cook, "Birds of a feather: Homophily in social networks," *Annu. Rev. Sociol.*, vol. 27, no. 1, pp. 415–444, 2001.
[71] Z. Yang, W. W. Cohen, and R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," 2016, *arXiv:1603.08861*.
[72] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. Vishwanathan, A. J. Smola, and H.-P. Kriegel, "Protein function prediction via graph kernels," *Bioinformatics*, pp. 47–56, 2005.
[73] P. D. Dobson and A. J. Doig, "Distinguishing enzyme structures from non-enzymes without alignments," *J. Mol. Biol.*, vol. 330, no. 4, pp. 771–783, 2003.
[74] N. Shervashidze, S. Vishwanathan, T. Petri, K. Mehlhorn, and K. Borgwardt, "Efficient graphlet kernels for large graph comparison," in *Proc. 12th Int. Conf. Artif. Intell. Statist.*, 2009, pp. 488–495.
[75] K. M. Borgwardt and H.-P. Kriegel, "Shortest-path kernels on graphs," in *Proc. 5th IEEE Int. Conf. Data Mining*, 2005.
[76] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," 2019, *arXiv:1903.02428*.
[77] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
[78] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[79] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *J. Mach. Learn. Res.*, vol. 11, pp. 2837–2854, 2010.
[80] A. J. Gates and Y.-Y. Ahn, "The impact of random models on clustering similarity," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 3049–3076, 2017.
[81] L. v. d. Maaten and G. Hinton, "Visualizing data using T-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.



Zhen Zhang received the BS degree in network engineering from Qingdao University, China, in 2015. He is currently working toward the PhD degree with the College of Computer Science, Zhejiang University. His research interests include graph analysis, recommender system, and machine learning.



Jiajun Bu (Member, IEEE) received the BS and PhD degrees in computer science from Zhejiang University, China, in 1995 and 2000, respectively. He is currently a professor with the College of Computer Science, Zhejiang University. His research interests include intelligent media computing, data mining, and information accessibility.



Martin Ester (Senior Member, IEEE) received the PhD degree in computer science from ETH Zurich, Switzerland, in 1989. Since November 2001, he has been an associate professor and is currently a full professor with the School of Computing Science, Simon Fraser University. His current research interests include social network analysis, recommender systems, biological network analysis, and data mining.



Jianfeng Zhang received the BS degree from Zhejiang University. He is currently the head of Alibaba DAMO Academy. His research interests include deep learning, data mining, and cloud computing.



Zhao Li received the PhD degree from Computer Science Department, University of Vermont. He is currently a senior staff scientist with Alibaba Group. His research interests include e-commerce ranking, recommendation systems, network representation learning, multiagent reinforcement learning, and big data-driven security.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.



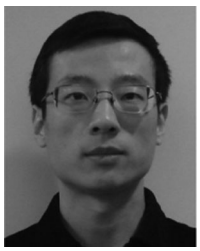
Chengwei Yao received the BS and PhD degrees in computer science from Zhejiang University, China, in 2000 and 2017, respectively. His research interests include machine learning, text mining, and recommender system.



Huifen Dai received the BS degree from Wenzhou Medical College, China, in 1985. Her research interests include machine learning and AI + medicine.



Zhi Yu received the PhD and BS degrees in computer science from Zhejiang University, China, in 2016 and 2010, respectively. He is currently an assistant researcher with the College of Software Technology, Zhejiang University. His research interests include information accessibility, data mining, and machine learning.



Can Wang (Member, IEEE) received the BS degree in economics, and the MS and PhD degrees in computer science from Zhejiang University, in 1995, 2003, and 2009, respectively. He is currently an associate professor with the College of Computer Science, Zhejiang University, China. His research interests include data mining, machine learning, and information retrieval.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Chulalongkorn University provided by UniNet. Downloaded on July 14,2026 at 17:11:35 UTC from IEEE Xplore. Restrictions apply.